

Claude Code — Extension Features Overview

Summary of <https://code.claude.com/docs/en/features-overview.md>

What Are Extensions?

Claude Code's built-in tools handle file operations, search, execution, and web access. Extensions go further, letting you customize what Claude knows, connect it to external services, and automate workflows.

Extension Feature Summary

Feature	What it does	Best for
CLAUDE.md	Persistent context loaded every session	Project rules, always-on conventions
Skill	Instructions, knowledge & workflows	Reusable docs, repeatable tasks
Subagent	Isolated execution returning a summary	Context isolation, parallel tasks
Agent Teams	Multiple independent Claude Code sessions	Parallel research, competing hypotheses
MCP	Connect to external services	DB queries, Slack, browser control
Hook	Deterministic script on events	Linting, logging, no LLM involved
Plugin	Packages skills, hooks, subagents, MCP	Reuse & distribute across repos

CLAUDE.md

- Loads automatically at every session start — full content every request.
- Multiple CLAUDE.md files are additive: project, user, and subdirectory levels all contribute.
- Keep it under 200 lines. Move reference material to Skills to avoid context bloat.
- Use `.claude/rules/` files for path-specific or language-specific guidelines.

Skills

- Markdown files containing knowledge, workflows, or instructions.
- Descriptions load at session start (low cost); full content loads only when invoked.
- Invoke manually with `/skill-name` or let Claude load them automatically when relevant.
- Set `disable-model-invocation`: true to hide from Claude until you call it — zero context cost.
- Bundled skills included: `/simplify`, `/batch`, `/debug`.

Subagents

- Run in an isolated context window; only the summary returns to your main session.

- Don't inherit conversation history — you must pass skills explicitly via the skills: field.
- Ideal for tasks reading many files or doing extensive searches without polluting main context.
- Can be pre-loaded with specific skills at launch.

Agent Teams (experimental)

- Multiple independent Claude Code sessions that communicate peer-to-peer.
- Teammates share a task list and can message each other directly.
- Higher token cost than subagents — each teammate is a separate Claude instance.
- Best for complex work: parallel code review, competing hypotheses, new feature development.
- Disabled by default; must be explicitly enabled.

MCP (Model Context Protocol)

- Connects Claude to external services: databases, Slack, browsers, APIs.
- Tool names load at session start; full JSON schemas are deferred until needed.
- Uses tool search by default, keeping idle MCP tools at minimal context cost.
- Scope hierarchy: local > project > user.
- Check /mcp to see token costs per server and disconnect unused ones.

Hooks

- Deterministic scripts triggered by lifecycle events (file edits, session start, etc.).
- Run entirely outside the LLM loop — zero context cost unless they return output.
- Perfect for side effects: linting after edits, logging, Slack notifications.
- All hooks for a matching event fire regardless of source (they merge, not override).

Plugins & Marketplaces

- Plugins bundle skills, hooks, subagents, and MCP servers into one installable unit.
- Plugin skills are namespaced (e.g. /my-plugin:review) to avoid conflicts.
- Use plugins to reuse a setup across repos or share with others.
- Marketplaces let you host and distribute plugin collections.

Context Cost at a Glance

Feature	When it loads	Context cost
CLAUDE.md	Session start	High — every request
Skills (default)	Start + on use	Low — descriptions only at start

Skills (user-only)	On manual invocation	Zero until invoked
MCP servers	Session start	Low — schemas deferred
Subagents	On demand	Isolated from main session
Hooks	On trigger	Zero (external scripts)

Combining Features

Skill + MCP: MCP provides the connection; a skill teaches Claude how to use it well (e.g. your DB schema + query patterns).

Skill + Subagent: A skill spawns subagents for parallel, isolated work (e.g. /audit kicks off security, perf, and style workers).

CLAUDE.md + Skills: CLAUDE.md holds always-on rules; skills hold reference material loaded only when needed.

Hook + MCP: A post-edit hook sends a Slack notification through an MCP server when critical files change.